

Platform Bring-Up Guide

Bringing new platforms to FujiNet on the prototype board — the design, the decisions, and two real examples.

Worked examples: MSX and the Tandy Color Computer

About this guide

This manual helps a small team of hardware and firmware engineers bring a new computer onto FujiNet using the **bus-interface tandem** design — a Raspberry Pi RP2350 that speaks the machine’s native bus, paired with an ESP32 that runs the FujiNet device firmware — built on the project’s **prototype board** ([fujiversal-pcb-prototype](#)). It is deliberately not a recipe. It explains **why the board is designed the way it is**, then works two real bring-ups — MSX and the Tandy Color Computer — so you can see the decisions and adapt them. The aim is to teach reasoning, not steps, because reasoning is the only thing that transfers to the machine on your bench.

Every register value, pin assignment, packet field, and source excerpt in this guide was transcribed from the live project sources listed on the copyright page of each repository, not from secondary documentation. Where the prototype is deliberately incomplete or risky, the text says so plainly.

Canonical sources

fujinet-bringup	START HERE – minimal byte relay + <code>iotest</code> host test; the bring-up MVP
fujiversal	RP2350 bus-interface firmware – PIO, USB-CDC bridge, ROM emulation
fujiversal-pcb-prototype	Universal proto board, ISA / CoCo / MSX adapters, footprints
fujinet-lib-experimental	Host client – FujiBus framing over the I/O byte pipe
fujinet-firmware	ESP32 device firmware – lib/bus, lib/device, lib/media
fujinet-config	Host-side ROM / CONFIG image served by the RP2350
FEP-004 (wiki)	FujiNet serial-encapsulation protocol proposal

Contents

1	Introduction	3
1.1	Start at fujinet-bringup	3
1.2	Choosing the interface: ESP32 or RP2350	4
1.3	The two decisions	4
1.4	The division of labour	5
1.5	What you will build, and how this guide teaches it	5
1.6	Conventions	6
2	System architecture	7
2.1	The physical stack	7
2.2	Why the inter-board bus is shaped like an ISA slot	7
2.3	The two firmware images and one ROM image	7
2.4	The communication layers	8
3	The FujiBus protocol (FEP-004)	9
3.1	Framing: SLIP	9
3.2	The packet header	9
3.3	The checksum	9
3.4	Field descriptors and AUX	10
3.5	Devices and commands	10
3.6	Request and response	11
4	The prototype board, and how to think about it	13
4.1	Why the board exists	13
4.2	Why the header is shaped like an ISA slot	13
4.3	The default GPIO routing	13
4.4	The jumper farm is a routing decision, not a checklist	14
4.5	The breakout headers are also patch points	14
4.6	Customising the board for your bus	14
5	MSX: when the board fits	15
5.1	The decisions	15
5.2	Mapping MSX onto the board	15
5.3	The byte pipe and the boot ROM	15
5.4	The adapter	16
6	The Color Computer: when the board is not enough	17
6.1	The decision that sets the tone: cartridge, not DriveWire	17
6.2	Where the board comes up short	17
6.3	Four ways to add the missing connections	17
6.4	The fingerprints of a tight fit	18
6.5	The byte pipe and the boot ROM	18
7	Hardware decisions you will face	19
7.1	Voltage: usually nothing to do	19
7.2	The adapter, from patched header to PCB	19
7.3	Power	19
7.4	First power-on, in groups	19
8	Inside the fujiversal firmware	22
8.1	Two cores	22
8.2	The three PIO state machines	22

8.3	The BusSignals union	22
8.4	ROM emulation and the I/O window	23
8.5	ROM bank-switching: the one packet the RP2350 keeps	23
8.6	USB transport and SLIP	24
9	Writing the PIO for your bus	25
9.1	Pin defines and the BusSignals union	25
9.2	Decode: the <code>wait_sel</code> decision	25
9.3	Driving data: the <code>read</code> decision	26
9.4	Building the board into fujiversal	27
9.5	Generating the host ROM	27
10	Bringing the RP2350 up	28
10.1	Flash and enumerate	28
10.2	The loopback test	28
10.3	Debugging with the registers	28
11	Where a bus platform fits in fujinet-firmware	30
11.1	The rs232 bus is the FujiBus transport	30
11.2	What this means for your effort	30
12	Adding the platform to the build system	31
12.1	The build platform file	31
12.2	The pin map	31
12.3	Partitions and flashing	32
13	Device classes	33
13.1	What you inherit	33
13.2	The virtualDevice contract	33
14	Media classes	34
14.1	Reuse first	34
14.2	When you need a new media class	34
15	The host ROM and CONFIG	35
15.1	What fujinet-config produces	35
15.2	How the loader talks to FujiNet	35
15.3	Building and chaining the ROM	35
16	The client library	36
16.1	The backend pattern	36
16.2	What bus/	36
16.3	Building it	36
17	The bring-up milestone ladder	39
18	Troubleshooting	40
19	Design exercise: a bus nobody has built yet	42
19.1	The method: what changes from one bus to the next	42
19.2	Working the diff for ISA	42
19.3	What you would build next	43
A	FujiBus quick reference	44
B	ISA 8-bit (PC/XT) 62-pin pinout	45
C	Universal board jumper & test-point reference	46
D	Repository map	47
E	Glossary	48
F	Bill of materials (one bring-up rig)	49

PART I

Orientation

What it means to bring up a platform, how the tandem design divides the work between two chips, and the FujiBus protocol that ties them together. Read this part before touching hardware.

CHAPTER 1

Introduction

FujiNet is a network and storage peripheral for retro computers. To a 1980s machine it looks like a fast disk drive, a printer, an RS-232 modem, a real-time clock, and a handful of other devices; behind that façade it is an ESP32 microcontroller with WiFi, an SD card, and a small army of internet protocol adapters. “Bringing up a platform” means making FujiNet appear as those familiar peripherals on a machine it has never run on before.

1.1 Start at fujinet-bringup

IMPORTANT

Before anything in this guide, clone and read the `fujinet-bringup` repository. It is the project’s canonical, deliberately-minimal first step, and it exists so you can prove two-way communication with your machine **before** committing to PCBs, PIO programs, or ROM emulation. This guide is what comes after — the production tandem design — but `fujinet-bringup` is where the work actually begins.

`fujinet-bringup` contains three small pieces and one method:

iotest a tiny host-side program (it runs on your retro machine) that echoes bytes between the keyboard/screen and the bus. You port it by writing a `portio` for your platform.

esp32 / rp2350 minimal **byte-relay** firmware for the microcontroller — it does nothing but shuttle bytes between the host bus (on GPIO) and a USB serial port. No FujiNet logic at all.

The method get those two talking, then point the relay’s USB port at the FujiNet firmware running as a **PC build** and run a “Hello World” that asks FujiNet for its version. Only once that works do you graduate to the on-board, ROM-emulating design this guide details.

This matters because it inverts the risk. The hard, scary part of a bring-up is the electrical and timing layer; `fujinet-bringup` lets you nail that with a breadboard, a relay, and a terminal loop before a single custom board is fabricated.

1.1.1 The host-side contract: `portio`

Whichever path you take, the host always talks to FujiNet through five routines. This is the same contract `iotest`, this guide’s client library (Chapter 16), and the production firmware all share:

```
// the portio contract (fujinet-bringup iotest/src/<platform>/portio.*)
void    port_init(void);
bool    port_available(void);           // is a byte waiting?
int     port_getc(void);                // read one byte, or -1
int     port_getc_timeout(uint16_t ticks);
uint16_t port_getbuf(void *buf, uint16_t len, uint16_t timeout);
void    port_putc(uint8_t c);           // write one byte
uint16_t port_putbuf(void *buf, uint16_t len);
```

`iotest` already ships working `portio` examples and build makefiles for roughly a dozen platforms — `adam`, `apple2`, `atari`, `c64`, `coco`, `dragon`, `h89-cpm`, `msdos`, `msx`, `vic20`, and more — so for many machines you are adapting an example, not starting blank. The host loop itself is just:

```
// iotest/src/main.c (the entire two-way test)
port_init();
while (1) {
    if (kbhit())          port_putc(cgetc()); // key -> bus -> relay -> USB
    if (port_available()) putchar(port_getc()); // USB -> relay -> bus -> screen
}
```

1.2 Choosing the interface: ESP32 or RP2350

The first design decision `fujinet-bringup` asks you to make is **which microcontroller sits on the bus**, and it turns on one number: how many bus signal lines you must manage.

≤ 8 signal lines	ESP32 can do it	Few enough lines to bit-bang from the ESP32's GPIO. The ESP32 is not 5 V tolerant, so it needs a level translator (the <code>fujinet-bringup</code> H89 example drives a <code>74LVC245</code> via <code>OE / DIR</code>). The H89 reaches FujiNet through an i8255 PPI in this way.
> 8 signal lines	use an RP2350	Enough GPIO for a wide address/data/control bus, and — per <code>fujinet-bringup</code> — the RP2350 can interface to 5 V signal lines directly, without a level shifter . That capability is the main reason to reach for it.

Table 1. The interface decision. ISA, with 20 address + 8 data + several control lines, is firmly in RP2350 territory — which is why this guide's worked example uses one.

IMPORTANT

Note the 5 V point, because it corrects a natural assumption (and an error in this guide's first edition): the RP2350's direct connection to a 5 V bus is **intentional and supported**, not a hazard to be buffered away. Level shifting in this design is an **ESP32** concern, not an RP2350 one. See Chapter 7.

1.3 The two decisions

Resist the urge to frame a bring-up as “which existing bus do I copy?” That is the wrong question and it leads to brittle designs. There are really only **two** decisions about how FujiNet talks to a machine, sitting underneath one overriding goal.

The goal: look like a boot device. The best user experience asks the least of the user — power on, and FujiNet is simply **there**. No second peripheral to own, no driver to side-load first, no disk to boot from something else. So wherever the machine allows it, FujiNet should **look like a normal boot device** and come up from bare metal. Hold this goal in mind; it decides close calls.

Decision 1 — do you connect through an existing disk interface, or not?

No (the common case) FujiNet speaks **FEP-004** (Chapter 3) directly, over whatever transport the machine offers — a serial port, or a microcontroller you place on the parallel expansion bus. Cartridge ports and card slots fall here, and this is the path the rest of the guide builds.

Yes Some machines already have a peripheral **disk** interface with a documented protocol — Atari's SIO, the Tandy CoCo's DriveWire. FujiNet can ride it by presenting as a drive, and the wire protocol could even be **FEP-004 carried inside** the disk protocol. But “could” is not “should”: whether it makes sense depends entirely on the disk protocol, and you cannot judge that without understanding it first. Studying that protocol is the job in this case.

Decision 2 — which microcontroller sits on the bus — is the electrical question from the previous section: count your signal lines (ESP32 for few, RP2350 for many), and remember the RP2350 takes 5 V directly.

NOTE

The two decisions are independent, and the UX goal often breaks the tie. Riding a disk interface can be less work, but it frequently costs the bare-metal-boot experience: the user may have to own a disk controller or load its software before FujiNet appears. A cartridge or card slot, by contrast, lets FujiNet present a boot ROM and come up on its own. When in doubt, favour the path that boots from bare metal.

When the answer to Decision 1 is “no disk interface” and the transport is a parallel expansion bus, you cannot simply hang the ESP32 on that bus: parallel CPU buses are **fast and unforgiving**. A Z80 or an 8088 expects valid data within tens of nanoseconds of asserting a read strobe; an ESP32 running FreeRTOS and a WiFi stack cannot meet that deadline. An RP2350 can — its Programmable I/O (PIO) blocks are deterministic state machines that react in single clock cycles, and a whole core can be dedicated to the bus. That is why a wide parallel-bus platform uses the two-chip design described next, and why this guide — and the prototype board it is built around — exists.

1.4 The division of labour

The single most important idea in this guide is how responsibility is split across the two chips. Internalise this and the rest of the manual is detail.

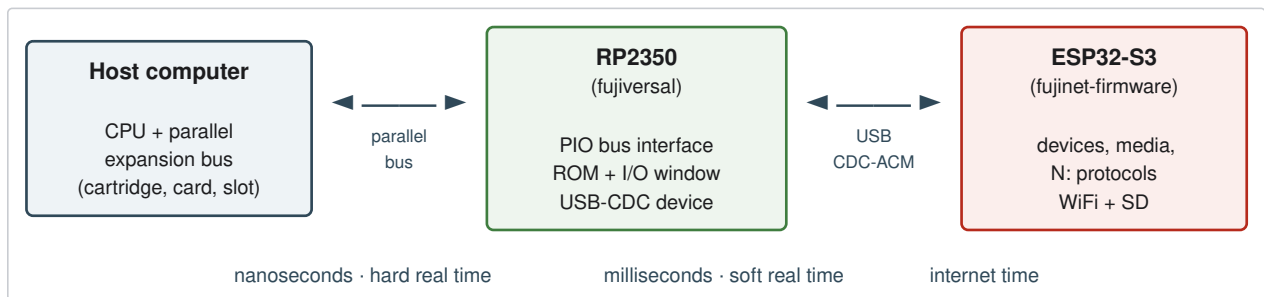


Figure 1. The tandem architecture. The RP2350 owns the time-critical parallel bus; the ESP32 owns devices and the network. They meet over a USB serial link carrying FujiBus packets.

- **The host** runs unmodified software. To it, FujiNet is a ROM in its address space plus a few I/O registers — nothing more exotic than a period-correct expansion card.
- **The RP2350** (the `fujiversal` firmware) emulates a ROM chip and a small bank of I/O registers on the host’s bus. The ROM holds the host-side loader and CONFIG program. The I/O registers are a **byte pipe**: the host pushes and pulls bytes through them, and the RP2350 shuttles those bytes over USB to the ESP32. The RP2350 understands almost nothing about FujiNet devices — it is a transparent pipe with one exception (ROM bank-switching, covered in Chapter 8).
- **The ESP32** (the `fujinet-firmware`) runs the entire FujiNet device stack: the Fuji control device, virtual disk drives, the `N`: network device, printers, clock, CP/M, and the internet protocol adapters behind them. It receives FujiBus packets over the USB serial link exactly as if they had arrived on any other FujiNet transport.

IMPORTANT

Because the ESP32 sees a **serial stream of FujiBus packets**, a bus-based platform reuses the firmware’s existing serial transport — the `rs232` bus class — almost unchanged. The genuinely platform-specific engineering concentrates in two places: the **RP2350 PIO program** (Part III) and the **host ROM + client library** (Part IV). The ESP32 side is mostly a build-system and pin-map exercise. This is the payoff of the tandem design, and a recurring theme of this guide.

1.5 What you will build, and how this guide teaches it

By the end of a bring-up you will have produced, for your target machine:

1. A configured **prototype board** — the right signals routed to the right GPIO for your bus, the two dev boards seated and powered (Part II).
2. A **bus adapter** — a small PCB (or, early on, a patched header) that mates the board to the machine's physical connector (Part II).
3. An **RP2350 PIO program** — a `boards/<platform>.pio` file that decodes your bus and implements the ROM + byte-pipe (Part III).
4. **ESP32 firmware support** — a build target, a pin map, and (only if needed) new device or media classes (Part IV).
5. A **host ROM and client library** — the loader/CONFIG image the RP2350 serves, and the `fujinet-lib` bus backend that drives the byte pipe (Part IV).

This guide does **not** hand you a recipe for one bus. Instead it explains the prototype board's design and walks two real bring-ups — **MSX** and **CoCo** — showing the decisions that were made and why. MSX is the case where the board fits the machine cleanly; CoCo is the case where it does not, and has to be coaxed. Study both and you will be equipped to reason about **your** machine, which is the only skill that actually ports. A short design exercise near the end (Chapter 19) applies that reasoning to a bus nobody has built yet.

1.6 Conventions

Inline code, signal names, register names, and file paths are set in `monospace`. Active-low signals are written `/IOR` or with an overline in schematics (KiCad renders these as `~{IOR}`). Hexadecimal is written `0x1234`; an I/O **port** address is also written `0x300`. GPIO numbers on the RP2350 are written `GP17`. Callout boxes carry four levels of emphasis:

NOTE

Background, rationale, or a pointer to a deeper source file.

TIP

A shortcut, a known-good value, or a debugging trick.

IMPORTANT

Something you must get right or the design will not work.

⚠️ MAGIC SMOKE — HARDWARE HAZARD

A way to destroy hardware. Read these twice.

CHAPTER 2

System architecture

This chapter is the map. It names every board, chip, firmware image, and wire, and shows where each one lives in the repositories. Later chapters zoom into the boxes drawn here.

2.1 The physical stack

A bring-up rig is three boards (Figure 2 shows how they relate):

Waveshare Core2350B the RP2350 board (component `U1` on the prototype schematic, footprint `FujiNet:WaveShare-Core-RP2350B`). The RP2350B variant brings out up to 48 GPIO, which is what makes a 20-bit address bus plus 8-bit data plus control signals fit on one chip. It connects to the host bus through the prototype board, and to the ESP32 over USB.

Freenove ESP32-S3-CAM the ESP32 board (component `U2`, footprint `FujiNet:ESP32-S3-CAM`). It runs the FujiNet firmware, holds the WiFi radio, and carries the microSD slot.

Universal prototype board `fujiversal-pcb-prototype/Bus-proto` (`Universal-proto-v1`). It seats both dev boards, distributes power, and routes every RP2350 GPIO to a generic **bus header** through a farm of solder jumpers. That header is an 8-bit ISA card-edge — see the next section for why.

Bus adapter a small per-machine board that converts the universal board's bus header into the target machine's physical connector. The repository already contains `CoCo-adapter` and `MSX-adapter`; building the **ISA adapter** is Chapter 6.

NOTE

The Freenove board may need a hardware tweak before its microSD works: one SD pin must be grounded by a solder bridge. The prototype README flags this (`<insert details about soldering microSD pin to ground here>`); confirm against your board revision before assembly.

2.2 Why the inter-board bus is shaped like an ISA slot

The prototype board's generic bus header uses the footprint of an 8-bit ISA edge connector (`Connector:Bus_ISA_8bit`). This is a **practical choice of a cheap, available 62-pin connector** whose signal set is a superset of most 8-bit buses — not a sign that ISA is the reference platform. Each per-machine adapter (`MSX-adapter`, `CoCo-adapter`) carries a matching `Bus_ISA_8bit` on the board side and the real machine connector (`MSX-Edge`, `CoCo-edge`) on the other. The board's **default** GPIO routing (Table 7) follows the ISA pin roles, but that is a starting assignment you customise per bus, not a fixed pinout — Chapter 4 is the full design rationale, and Chapters 5–6 show it bent to fit MSX and CoCo.

2.3 The two firmware images and one ROM image

Three pieces of code run in a working system:

<code>fujiversal</code> UF2	RP2350	<code>fujiversal/</code> — <code>make ROM_FILE=... BOARD=...</code>
FujiNet firmware	ESP32-S3	<code>fujinet-firmware/</code> — PlatformIO env
Host ROM	host CPU (served by RP2350)	<code>fujinet-config/</code> — per-platform loader + CONFIG

Table 2. The three build products. The host ROM is **data** compiled into the RP2350 image (`build/<board>/rom.h`), not a separate flash.

The host ROM deserves emphasis because it is easy to forget. The RP2350 does not invent the bytes the host CPU fetches from the emulated ROM; those bytes come from `fujinet-config` , which builds a small loader and the CONFIG user interface for each platform. You build that ROM image **first**, then build `fujiversal` with `ROM_FILE` pointing at it (Chapter 15).

2.4 The communication layers

From the host CPU down to the internet, a single disk read passes through five distinct layers. Knowing which layer owns a problem is the difference between an hour and a week of debugging.

	Layer	Owner	Responsibility
5	Device / media	ESP32	Fuji, disk, N:, printer, clock; image formats; N: protocol adapters
4	FujiBus (FEP-004)	ESP32 ⇄ host lib	device + command + AUX fields + payload, SLIP-framed, checksummed
3	Byte pipe	RP2350 ⇄ host	the 4 I/O registers: GETC / STATUS / PUTC / CONTROL
2	Bus interface (PIO)	RP2350	ROM emulation, address decode, drive/sample the data bus
1	Physical bus	adapter + board	voltage levels, connector, timing, AEN / strobes

Figure 2. The five layers of a FujiNet transaction on a tandem platform. The chapters of this guide are organised bottom-up: Part II builds layer 1, Part III builds layers 2–3, Part IV builds layers 4–5.

CHAPTER 3

The FujiBus protocol (FEP-004)

FujiBus is the packet protocol the ESP32 and the host exchange. It is the working implementation of the FEP-004 proposal, and it is identical whether the bytes travel over a true RS-232 line, an MSX serial port, or — in our case — the USB-CDC link between the RP2350 and the ESP32. This chapter is a precise reference; you will implement an encoder for it in the client library (Chapter 16) and decode it in the PIO bridge’s control path (Chapter 8).

The authoritative implementations are `FujiBusPacket.cpp` (present in both `fujiversal/` and `fujinet-firmware/lib/bus/rs232/`) and the C encoder in `fujinet-lib-experimental`. Where the FEP-004 wiki draft left questions open, the live code answers them; this chapter documents the code.

3.1 Framing: SLIP

Every packet is wrapped in SLIP (RFC 1055) so the receiver can find frame boundaries in a raw byte stream. A frame **both starts and ends** with the `END` byte.

END	0xC0	Frame delimiter (sent before and after every frame)
ESC	0xDB	Escape prefix
ESC_END	0xDC	Follows ESC to mean a literal 0xC0
ESC_ESC	0xDD	Follows ESC to mean a literal 0xDB

Table 3. SLIP framing bytes (`FujiBusPacket.h`). Any `0xC0` in the payload is sent as `0xDB 0xDC` ; any `0xDB` as `0xDB 0xDD` .

3.2 The packet header

Inside the SLIP frame is a fixed six-byte header followed by optional field descriptors, optional AUX fields, and an optional payload. All multi-byte values are **little-endian**.

device 1	command 1	length 2 (LE)	checksum 1	descr 1	fields... AUX	payload...
0	device	Destination device ID (see device table below)				
1	command	Command ID; in a reply, <code>ACK</code> (<code>0x06</code>) or <code>NAK</code> (<code>0x15</code>)				
2–3	length	Total decoded packet length including the header, little-endian				
4	checksum	8-bit add-with-carry-fold over the whole packet with this byte zeroed				
5	descr	First field descriptor (see below); <code>0x00</code> if there are no AUX fields				

Table 4. The six-byte FujiBus header (`struct fujibus_header` , `FujiBusPacket.cpp`).

NOTE

The header struct is declared `static_assert(sizeof(fujibus_header) == 6)` and the checksum byte is asserted to be at offset 4. If you re-implement the header in another language, preserve this layout exactly — the firmware reads it as a packed C struct.

3.3 The checksum

The checksum is **not** a plain XOR. It is an 8-bit running sum that folds the carry back in after every byte, computed over the entire packet (header + descriptors + AUX + payload) with the checksum byte set to zero:

```
uint8_t calcChecksum(const ByteBuffer &buf) {
    uint16_t chk = 0;
    for (size_t i = 0; i < buf.size(); ++i) {
        chk += buf[i];
        chk = (chk >> 8) + (chk & 0xFF);    // fold carry
    }
    return (uint8_t) chk;
}
```

3.4 Field descriptors and AUX

Most FujiNet commands carry a few small “AUX” arguments — a disk sector number, a network open mode, a string length. FEP-004 packs these compactly. The `descr` byte (and any additional descriptor bytes) encodes how many AUX values follow and how wide each is.

descr			FUJI_FIELD_*
0	0	—	NONE
1	1	1 byte	A1
2	2	1 byte	A1_A2
3	3	1 byte	A1_A2_A3
4	4	1 byte	A1_A2_A3_A4
5	1	2 bytes	B12 (a uint16)
6	2	2 bytes	B12_B34 (two uint16)
7	1	4 bytes	C1234 (a uint32)

Table 5. Field-descriptor encoding. The two lookup tables in the code are `numFieldsTable = {0,1,2,3,4,1,2,1}` and `fieldSizeTable = {0,1,1,1,1,2,2,4}`.

Bit 7 of a descriptor (`FUJI_DESCR_ADDTL_MASK` , `0x80`) means “another descriptor byte follows”, letting a packet mix field widths (for example a 32-bit sector number **and** a one-byte unit). AUX values are written little-endian immediately after all descriptor bytes; anything left over is the payload.

NOTE

You rarely hand-assemble descriptors. The client library exposes `fuji_bus_call(device, cmd, fields, a1, a2, a3, a4, data, len, reply, reply_len)` plus a family of `FUJICALL_*` macros (`FUJICALL_A1` , `FUJICALL_B12_D` , `FUJICALL_C1234` , ...) named for exactly these field codes. Chapter 16 shows the byte pipe underneath them.

3.5 Devices and commands

A packet’s first byte selects a device. The RP2350 also watches this byte: a packet addressed to device `0xFF` (`FUJI_DEVICEID_DBC` , the bus controller itself) is consumed by the RP2350 and never reaches the ESP32 (Chapter 8).

0x31 — 0x3F	DISK ... DISK_LAST	Virtual disk drives (block devices)
0x40 — 0x43	PRINTER ... PRINTER_LAST	Printers / voice
0x45	CLOCK	Real-time clock (APETime)
0x50 — 0x53	SERIAL	Serial / modem passthrough
0x5A	CPM	CP/M console
0x70	FUJINET	The Fuji control device (mounts, hosts, config)
0x71 — 0x78	NETWORK ... NETWORK_LAST	N: network units (8 of them)
0x99	MIDI	MIDI
0xFF	DBC	Bus controller (RP2350) — intercepted locally

Table 6. FujiNet device IDs (`fujiDeviceID.h`). The same header is shared, byte-for-byte, by the RP2350 firmware and the client library.

Commands are a single byte. Many are printable ASCII, a convention inherited from the Atari SIO and N: heritage: 'O' (0x4F) opens, 'R' (0x52) reads, 'W' (0x57) writes, 'S' (0x53) gets status, 'C' (0x43) closes. The Fuji control device uses a dense high range (0xD0 — 0xFF) for mounts, host slots, app-keys, hashing, and the rest. The complete list is `fujiCommandID.h`, reproduced in Appendix A.

3.6 Request and response

The exchange is strictly command/response over the byte pipe:

1. The host (via the client library) builds a packet, SLIP-encodes it, and streams it out through the `PUTC` register.
2. The ESP32 decodes it, dispatches to the addressed device, and acts.
3. The ESP32 replies with a packet whose `command` is `ACK` (0x06) on success — carrying any requested data as payload — or `NAK` (0x15) on failure.
4. The host polls `STATUS` for the `available` bit, then reads the reply bytes back through `GETC`.

NOTE

The FEP-004 wiki draft explicitly left “reply packet structure” and “how to signal data availability” as open questions. The shipping code resolves them: replies are **full FujiBus packets** (same header, checksum, optional payload), and availability is signalled by the `STATUS` register’s `available` bit in the byte pipe — no platform-specific interrupt line required. Document your platform’s behaviour the same way.

PART II

The Prototype Board, by Example

What the prototype board is, why it is built the way it is, and how to bend it to a new bus — taught through two real bring-ups: MSX, where the board fits cleanly, and the Color Computer, where it does not and has to be coaxed into shape.

CHAPTER 4

The prototype board, and how to think about it

Before any single platform, understand the board every platform is brought up on: `fujiversal-pcb-prototype/Bus-proto` (`Universal-proto-v1`). It is not an ISA card, not a CoCo cartridge, not an MSX cartridge — it is a **development fixture** designed so you can try any of those buses without spinning a new PCB each time. This chapter is about its design intent, because every later decision is a negotiation with that intent.

4.1 Why the board exists

Bringing a microcontroller up on an unfamiliar parallel bus is fiddly and iterative: you probe, you guess, you move a wire, you try again. Respinning a custom PCB for each guess would be absurd. The prototype board removes that cost. It does four things and nothing more:

1. It **seats the two dev boards** — the WaveShare Core2350B (`U1` , the RP2350) and the Freenove ESP32-S3-CAM (`U2`) — and wires the USB link between them.
2. It **distributes power** with a selectable source (`J9`) and reverse protection (`D1`).
3. It **routes every RP2350 GPIO** to a generic, bus-shaped header (`J1`) through a field of solder jumpers, so you choose what reaches the bus.
4. It **brings every signal out to 0.1" breakout headers** (`J2` on the bus side, `J3` / `J4` on the GPIO side) so you can probe — or patch — anything.

That is the whole board. There are **no buffers, no glue logic, no decode** — those are decisions left to you, because they depend on the bus.

4.2 Why the header is shaped like an ISA slot

The generic bus header `J1` has the footprint of an 8-bit ISA edge connector (`Connector:Bus_ISA_8bit`). This is a **practical convenience, not a statement that the board is an ISA card**. An 8-bit ISA edge is a cheap, widely-available 62-pin connector with a sane 0.1" pitch, and its signal set — twenty address lines, eight data lines, a handful of control strobes, and the common supply rails — is a comfortable **superset** of what most 8-bit buses need. So the board borrows that connector as its backplane, and the per-machine adapters (`MSX-adapter` , `CoCo-adapter`) carry a matching `Bus_ISA_8bit` on one side and the real machine connector on the other.

IMPORTANT

Do not read the ISA-shaped header as “ISA is the reference platform”. It is a generic interconnect that happens to use an ISA footprint. The worked examples in this guide are MSX and CoCo precisely to keep that distinction honest.

4.3 The default GPIO routing

Because the header is ISA-shaped, the board’s **default** jumper routing maps each GPIO to the ISA pin of the same role. Treat this as a starting assignment, not a fixed pinout: it is what the jumpers connect **until you decide otherwise**.

GP0 — GP7	A0 — A7	GP8 — GP15	A8 — A15	GP16 — GP19	A16 — A19
GP20 — GP27	D0 — D7	GP28	strobe 1	GP29	strobe 2
GP30	strobe 3	GP31	strobe 4	GP32	select / AEN
GP33	clock	GP34	latch / ALE	GP35	reset

Table 7. The board’s **default** GPIO routing (net labels `GP0_ISA_A0 ... GP35_ISA_RESET`). `GP36 — GP47` are unassigned spares on the breakout headers. The four “strobe” pins are wired to ISA’s `/MEMR / /MEMW / /IOR / /IOW` by default; a different bus repurposes them (the MSX example does exactly that).

4.4 The jumper farm is a routing decision, not a checklist

Thirty-nine solder jumpers sit between the GPIO map and the header. **This is the board’s customisation mechanism, and it is the part people misuse.** The jumpers are not a ritual to perform — you do not “cut them all and bridge them back”. You make a **per-signal routing decision**:

JP1 — JP36	bridged	Each connects one GPIO to its default header pin. Leave bridged the signals your bus uses in their default role; cut the ones it does not, to free that GPIO or that header pin for another use.
JP37 — JP39	open	Optional straps for alternate routing. Bridge one only when a specific need calls for it.

Table 8. How to think about the jumpers. For a bus that matches the default map you change almost nothing; for one that does not, you cut what you are repurposing and patch in what you need.

The practical consequence is the design question you should be asking from the start: **which of my bus’s signals fall on the default GPIO, which do not, and how will I bring the strays in?** The MSX and CoCo examples are the two answers to that question — “almost all of them do” and “several of them don’t”.

4.5 The breakout headers are also patch points

`J2` (every bus-side signal) and `J3 / J4` (every GPIO) are the obvious logic-analyzer taps, and you will live on them during bring-up. But they have a second, less obvious job: they are how you **add** a connection the board does not route by default. If your bus needs a signal on a GPIO the default map never wired to the header, you do not respin the board — you run a jumper wire from the GPIO breakout to wherever it needs to go. The CoCo example turns on exactly this.

4.6 Customising the board for your bus

Putting the chapter together, customising the board for a new machine is a sequence of decisions, not a procedure:

1. **Enumerate the bus signals** the interface must see (address, data, selects, strobes, clocks, reset — and any oddities).
2. **Match them to the default GPIO routing.** Which land where they already are? Which need a jumper cut and re-bridged elsewhere? Which are not on the header at all and must be patched from a breakout?
3. **Decide the byte-pipe and ROM addresses** — where can the host reach four registers, and is there an address window you can present a boot ROM in?
4. **Decide voltage and power** — almost always “nothing to do” on an RP2350 (Chapter 7).

Hold those four decisions in mind through the next two chapters; they are the spine of both worked examples.

CHAPTER 5

MSX: when the board fits

The MSX bring-up is the easy case, and the right one to study first, because the prototype board’s default routing nearly matches the machine. Watch how few decisions are actually forced.

5.1 The decisions

- **Disk interface? No.** The MSX has cartridge and expansion slots but no standard peripheral **disk** protocol to ride, so FujiNet speaks **FEP-004** directly (Decision 1 from Chapter 1).
- **Look like a boot device? Yes, and MSX makes it easy.** An MSX cartridge whose ROM begins with the signature bytes `0x41 0x42` (“AB”) at address `0x4000` is found by the BIOS at power-on, which calls the cartridge’s `INIT` entry. So FujiNet presents a ROM with that header and is a normal, auto-starting cartridge — bare-metal boot, nothing pre-installed. This is the UX goal met for free by the platform’s own conventions.
- **Which microcontroller?** The cartridge bus is wide (16 address + 8 data + several control), so RP2350 (Decision 2), which also takes the 5 V cartridge bus directly.

5.2 Mapping MSX onto the board

The MSX cartridge is a Z80 bus: `A0 – A15`, `D0 – D7`, `/SLTSL` (slot select), `/RD`, `/WR`, `/IORQ`, `/MERQ`, `CLOCK`, `RESET`. The board’s default routing absorbs it almost untouched — and where it does not match exactly, the four “strobe” jumpers are simply **repurposed** rather than rerouted:

<code>A0 – A15</code>	<code>GP0 – GP15</code>	Default address routing, unchanged.
<code>D0 – D7</code>	<code>GP20 – GP27</code>	Default data routing, unchanged.
<code>/RD</code> , <code>/WR</code> , <code>/IORQ</code> , <code>/MERQ</code>	<code>GP28 – GP31</code>	The four default “strobe” pins, repurposed from ISA’s <code>/MEMR</code> / <code>/MEMW</code> / <code>/IOR</code> / <code>/IOW</code> to the Z80’s four control lines. No rewiring — just a different meaning in the PIO.
<code>/SLTSL</code>	<code>GP32</code>	The default select pin (ISA’s <code>AEN</code>), now the cartridge slot-select.
<code>CLOCK</code>	<code>GP33</code>	The default clock pin.
<code>RESET</code>	<code>GP35</code>	The default reset pin.

Table 9. MSX on the prototype board (from `boards/msx_proto_260402.pio`). Nothing is cut or patched; the default routing carries the whole bus, and the PIO simply reads the four strobe pins as Z80 control lines.

That is what “the board fits” means: the only adaptation is in firmware (what the PIO calls each pin), not in copper.

5.3 The byte pipe and the boot ROM

MSX memory-maps the byte pipe rather than using an I/O port. The four registers live at `0xBFFC – 0xBFFF` — a free spot high in the cartridge’s page-2 window — and the boot ROM occupies `0x4000 – 0xBFFF`:

```
IO_BASE      0xBFFC      ; GETC=+0  STATUS=+1  PUTC=+2  CONTROL=+3
IO_FLAG_AVAIL 0x80       ; STATUS bit 7 set when a byte is waiting
BUS_ROM_BASE 0x4000      ; the cartridge ROM window (AB header lives here)
```

NOTE

`0xBFFC` is not arbitrary, and it is worth seeing the consistency: the `fujinet-bringup iotest MSX portio` reads and writes the **same** `0xBFFC` byte pipe (`IO_OFFSET = 0x8000 + 0x3FFC`). The relay MVP and the production board expose the identical four registers at the identical address — proof that the byte pipe is one contract, not two.

5.4 The adapter

Because the mapping is clean, the `MSX-adapter` is mostly straight wiring: an MSX cartridge edge on one side, a `Bus_ISA_8bit` on the other, signal for signal. The RP2350 meets the 5 V bus directly. There is little to **decide** here — which is exactly why MSX is the example to cut your teeth on.

CHAPTER 6

The Color Computer: when the board is not enough

The CoCo bring-up is the instructive case, because the prototype board, as designed, **does not have enough of the right signals connected** for it. Everything interesting about a real bring-up — a UX judgement call, a hardware shortfall, and four different ways to fix it — shows up here.

6.1 The decision that sets the tone: cartridge, not DriveWire

The CoCo is unusual in that it **does** have a disk interface FujiNet could ride: **DriveWire**, a serial disk protocol, which is how FujiNet's existing CoCo support works. So Decision 1 has a real “yes” answer available. Why does the prototype-board bring-up choose the cartridge port instead?

Because of the boot-device goal. Riding DriveWire means the user must have Disk BASIC and a DriveWire setup in place before FujiNet appears — a peripheral and pre-installed software, exactly what the UX goal says to avoid. The cartridge port, by contrast, **autostarts a Program Pak**: on power-up the Color BASIC ROM hands control to a cartridge at `0xC000`, so a FujiNet cartridge comes up from bare metal with nothing installed. The worse-effort path wins because it is the better experience. This is the Chapter 1 trade-off made concrete.

6.2 Where the board comes up short

The 6809 cartridge edge brings signals the ISA-shaped default routing never anticipated: **two** phase clocks (`E` and `Q`, where ISA has one), plus cartridge control lines — `/CART`, `/SLENB`, `/HALT`, `/NMI`, `SND` — that have no analogue in the default map at all. The board simply does not route all of them to where the `CoCo-adapter` can reach them. Put plainly: **as built, the proto board does not have enough signals connected for CoCo.**

This is not a defect; it is the expected outcome of a generic fixture meeting a specific machine. The skill is recognising the gap and closing it deliberately.

6.3 Four ways to add the missing connections

When a signal the bus needs is not routed to where it must go, you have a genuine design choice. None of these is “correct”; they trade reworkability against permanence against effort:

Breakout jumpers	Run a jumper wire from a <code>J3 / J4</code> GPIO breakout pin to where the signal must go. This is how the team manages CoCo today.	You are still iterating; you want to move it tomorrow
Solder front	Omit those breakout headers and solder a wire directly between the two pads.	The routing is settled and you want it low-profile
Solder back	Run the wire on the back of the board.	The front is crowded; mechanical clearance matters
Wire-wrap	Fit long wire-wrap posts and wire-wrap the connections.	Many signals to patch; you value reworkable-but-robust

Table 10. Four ways to patch a connection the board does not route. The CoCo bring-up uses breakout jumpers; the others are equally valid depending on how permanent and how dense the rework is.

The point of listing them is not to pick a winner — it is to show that “the board doesn’t have the signal” is a solvable problem with a spectrum of answers, and which you choose is a judgement about your build, not a rule.

6.4 The fingerprints of a tight fit

When the board does not match the machine, the firmware shows it. Two artefacts in `boards/coco_proto_260402.pio` are worth recognising, because you will produce your own versions of them:

1. **Relocated pins.** The file still carries the original pin assignment as a comment — `CTS=16` , `SCS=17` , `CLOCK=18` — above the assignment actually used — `CTS=32` , `SCS=30` , `CLOCK=33` . The signals were moved to whatever GPIO could actually be reached after patching, not to where they were first drawn.
2. **A defensive pull-up.** One unused middle pin (`GP31`) is pulled up in firmware “to avoid a false zero” on the bus. A stray, floating, or repurposed line often needs a small defensive measure like this; budget for one or two.

6.5 The byte pipe and the boot ROM

The CoCo places its byte pipe in I/O-ish space and its ROM in the cartridge window:

```
IO_BASE      0xFF41      ; STATUS=+0  GETC=+1  CONTROL=+2  PUTC=+3  (note order)
IO_FLAG_AVAIL 0x02      ; STATUS bit set when a byte is waiting
BUS_ROM_BASE 0xC000      ; the /CTS cartridge ROM window (autostart vector here)
```

The register **order and the available-bit position differ from MSX** — they are chosen to fall on convenient bits and addresses for the 6809 code, and that is fine: the byte pipe is a contract about four registers, not about their exact offsets. Timing references the `E` clock and `R/W` (a 6809 has no separate read and write strobes), which is the other reason the CoCo PIO looks different from the MSX one.

CHAPTER 7

Hardware decisions you will face

The two examples cover the routing decisions. This chapter collects the remaining hardware choices — voltage, the adapter, power, and how to bring a board up without flailing — and frames each as a decision rather than a step.

7.1 Voltage: usually nothing to do

On an RP2350 board this is the shortest decision in the guide: the RP2350 interfaces to a 5 V bus **directly** (Chapter 1), so there is no level shifter and no “5 V problem”. Both worked examples connect straight to the 5 V cartridge bus.

A translator becomes **mandatory** only on the narrow-bus **ESP32** path, which is not 5 V tolerant — the `fujinet-bringup` H89 example drives a `74LVC245` from the ESP32 via its `OE / DIR` pins for exactly this reason. On an RP2350 you would add buffering only for **signal integrity** on a long or heavily-loaded bus, never for protection.

7.2 The adapter, from patched header to PCB

“Adapter” is a spectrum, not a deliverable you need up front:

Bench bring-up	Jumper wires from the breakout headers to the machine’s connector. No PCB at all.
Settling	A scrap of perfboard, or wire-wrap, fixing the routing you proved on the bench.
Reproducible	A small PCB — machine edge on one side, <code>Bus_ISA_8bit</code> on the other — like the <code>MSX-adapter</code> and <code>CoCo-adapter</code> .

Table 11. You do not need the PCB to start. The examples’ adapters are the end state, not the entry point.

7.3 Power

Bring `+5V` and `GND` from the machine to the board’s power header through `J9`, which selects the source; `D1` blocks back-feed. The Freenove ESP32 module may need its microSD pin grounded by a solder bridge before the card works (the prototype README flags this) — confirm against your revision.

CAUTION

Decide the power source **before** connecting to a live machine. During bench bring-up, power from USB and leave the machine’s `+5V` disconnected so you are never tying two supplies together. Switch to bus power only once the interface is otherwise proven.

7.4 First power-on, in groups

Bring a board up in layers so a fault is contained to the layer you just added — and note what this is **not**: it is not “cut every jumper first”. For a clean-fit bus (MSX) you change no jumpers at all; for a tight-fit bus (CoCo) you first make your deliberate routing changes — cut the defaults you are repurposing, patch in the strays (the four ways above) — and only then stage the bring-up. Either way the staging is about **observability**:

1. **RP2350 alone**, on USB, no bus: confirm it enumerates as a USB CDC device (Chapter 10). Nothing should warm up.
2. **ESP32 alone**: flash it, confirm WiFi and SD.
3. **Bus, address + select**, watched at the breakouts: confirm the PIO sees the machine address the interface, and only when it should.

4. **Bus, strobes + data:** run the loopback of Chapter 10.
5. **In the machine:** only now move from the bench to a real slot and switch **J9** to bus power.

TIP

Keep a powered USB hub with per-port power between your workstation and both dev boards. You will reflash each many times, and per-port power lets you cycle one without disturbing the other or the bus.

PART III

The RP2350 Bus Interface

The `fujiversal` firmware: how two cores and three PIO state machines turn a generic dev board into a ROM-and-registers peripheral, and how to write the PIO program that decodes **your** bus. This is the most platform-specific code you will write.

CHAPTER 8

Inside the fujiversal firmware

`fujiversal` is a single Pico-SDK application that emulates a ROM chip and a four-register I/O window on the host's bus, and bridges that window to the ESP32 over USB. Before you write a line of ISA PIO, understand how the existing MSX and CoCo builds are structured, because your build is the same skeleton with a new `.pio` file.

8.1 Two cores

The firmware pins the time-critical work to one core and the housekeeping to the other.

Core 1 — `romulan()` the bus loop. It runs `__time_critical_func`, sets up the PIO, and then spins forever pulling latched bus words from a PIO FIFO and responding to them. It is the only code allowed to touch the bus. It services ROM reads and the I/O registers with no operating system, no allocation, and no blocking.

Core 0 — `main()` the USB bridge. It runs TinyUSB, moves bytes between the host (via core 1) and the ESP32 (via USB-CDC), maintains the SLIP framing, and intercepts the handful of packets addressed to the bus controller itself. It also feeds the watchdog.

The two cores exchange single bytes through the SDK's multicore FIFO, buffered on each side by a 1 KB ring.

8.2 The three PIO state machines

A bus-based board uses three PIO programs, set up in `setup_pio_irq_logic()`. They are defined per board in `boards/<board>.pio` and are the part you rewrite for a new bus.

<code>wait_sel</code>	Watch the select/decode signals. When the host addresses the card, raise an IRQ. This program encodes your bus's decode rule .
<code>send_bus</code>	On that IRQ, sample all 32 low GPIO in one instruction and autopush the 32-bit word to the RX FIFO. Core 1 reads it as a <code>BusSignals</code> union.
<code>read</code>	Drive <code>D0-D7</code> with a byte core 1 supplies, manage the data-pin direction (high-Z except during a read of our address), and release the bus when the cycle ends.

Table 12. The three PIO state machines. `wait_sel` and `read` are where bus timing lives; `send_bus` is essentially boilerplate.

8.3 The BusSignals union

`send_bus` hands core 1 a packed 32-bit word. A C union maps the bit positions of the GPIO onto named fields. This is the MSX definition; note how `addr:16`, `data:8`, and the control bits line up with the pin `.define`s in the same file:

```
typedef union {
    struct {
        uint32_t addr:16;    // A0..A15  on GP0..GP15
        uint32_t resv:4;
        uint32_t data:8;     // D0..D7   on GP20..GP27
        uint32_t rd:1;       // /RD
        uint32_t wr:1;       // /WR
        uint32_t iorq:1;     // /IORQ
        uint32_t memrq:1;    // /MEMRQ
    } __attribute__((packed));
}
```

```
uint32_t combined;
} __attribute__((packed)) BusSignals;
```

Your ISA union (Chapter 9) is wider in the address field and carries different control bits, but the idea is identical: the bit layout is the GPIO map.

8.4 ROM emulation and the I/O window

Core 1's loop is short enough to read in full. It decides, for every latched bus cycle, whether the address falls in the I/O window or the ROM window, and acts:

```
// from romulan(), main.cpp – the per-cycle decode
if (IO_BASE <= bus.addr && bus.addr < IO_TOP) {
    unsigned io_reg = (bus.addr - IO_BASE) & 0x3;
#ifdef RW_PIN
    if (!bus.rw) io_reg |= 2;          // fold R/W into the register index
#endif
    switch (io_reg) {
        case IO_GETC: pio_put_fifo(PSM_READ, sio_hw->fifo_rd); break;    // byte from
ESP32
        case IO_STATUS: pio_put_fifo(PSM_READ,
                                   sio_hw->fifo_st & SIO_FIFO_ST_VLD_BITS
                                   ? IO_FLAG_AVAIL : 0x00); break;        // is a byte
ready?
        case IO_PUTC:  sio_hw->fifo_wr = bus.combined; break;            // byte to ESP32
        case IO_CONTROL: break;
    }
}
else if (BUS_ROM_BASE <= bus.addr && bus.addr < BUS_ROM_TOP) {
    rom_offset = bus.addr - BUS_ROM_BASE;
    pio_put_fifo(PSM_READ, rom_ptr[rom_offset]);          // serve a ROM
byte
}
```

Two things to take from this:

1. The four registers are nothing more than the two ends of the multicore FIFO (`sio_hw->fifo_rd` / `fifo_wr`) dressed up as bus-visible ports. `GETC` pops a byte the ESP32 sent; `PUTC` pushes a byte toward the ESP32; `STATUS` exposes the FIFO's "valid" flag as the `available` bit.
2. `IO_BASE`, `IO_GETC`, `IO_STATUS`, `IO_PUTC`, `IO_CONTROL`, `IO_FLAG_AVAIL`, `BUS_ROM_BASE`, and `BUS_ROM_TOP` are all `.define`s in the board's `.pio` file. Re-pointing the byte pipe at ISA's address map is mostly a matter of changing those constants.

8.5 ROM bank-switching: the one packet the RP2350 keeps

There is a single exception to "the RP2350 is a transparent pipe". The host-side loader needs to swap the emulated ROM's contents at runtime (to page in CONFIG, or a booted disk's loader). It does this with FujiBus packets addressed to `FUJI_DEVICEID_DBC` (`0xFF`). Core 0 sniffs the second byte of every frame and, if it is `0xFF`, consumes the packet locally instead of forwarding it:

```
// main loop, core 0 – divert DBC frames to process_command()
if (command_size == 2 && input != FUJI_DEVICEID_DBC) {
    // second byte isn't 0xFF -> not for us, push to ESP32
    ...
}
```

```
else if (command_size > 1 && input == SLIP_END) {  
    process_command(command_buf);    // a complete DBC frame: handle locally  
}
```

`process_command()` implements a tiny device: `FUJICMD_OPEN` selects a RAM bank, `FUJICMD_WRITE` fills it, `FUJICMD_CLOSE` activates it (the next ROM fetch sees the new image), and `FUJICMD_RESET` reverts to the built-in ROM. You inherit this for free; an ISA build needs no change here.

8.6 USB transport and SLIP

Everything else core 0 does is plumbing: read a byte from USB (`tud_cdc_read`), and if a frame is in progress (started by a `SLIP_END`) accumulate it, otherwise pass the byte straight to core 1 for the host to read via `GETC` . Bytes the host writes via `PUTC` travel the other way, out through `tud_cdc_write` . A 50 ms timeout flushes a partial frame so a glitch cannot wedge the pipe. The USB side is `stdio` -based on some builds and raw TinyUSB CDC on others; either way the ESP32 sees a clean CDC-ACM serial port.

CHAPTER 9

Writing the PIO for your bus

The PIO is where a bus’s specifics live. Chapter 8 described the three state machines in the abstract; this chapter shows how the **real** MSX and CoCo `.pio` files fill them in, because the differences between those two files **are** the design decisions you will face. Read them side by side and the pattern generalises to any bus.

	<code>msx_proto_260402.pio</code>	<code>coco_proto_260402.pio</code>
What “selected” means	<code>/SLTSL</code> low — a ready-made slot-select line	<code>/CTS</code> or <code>/SCS</code> low — two separate cartridge selects you decode
Timing reference	none needed; the select line frames the cycle	the 6809 <code>E</code> clock edges
Read vs write	<code>/RD</code> / <code>/WR</code> (separate Z80 strobes)	a single <code>R/W</code> level
Data-direction flip	tri-state when the FIFO is empty or <code>/SLTSL</code> releases	flip <code>pindir</code> s with side-set around the <code>E</code> clock
Byte pipe	memory-mapped, <code>0xBFFC</code>	<code>0xFF41</code> in the <code>/SCS</code> I/O spot
ROM window	<code>0x4000</code> (AB-header cartridge)	<code>0xC000</code> (autostart Program Pak)

Table 13. The PIO decisions, read straight off the two shipping board files. The rest of this chapter walks the three that matter most.

9.1 Pin defines and the BusSignals union

Every board file opens with pin `.define`s that must match how the board is actually routed (Chapters 4–6) and a `BusSignals` union whose bit layout is that routing. They differ per bus: MSX is a clean 16-bit-address Z80 union; CoCo relocated several pins to whatever GPIO it could reach, so its defines do not sit on the default numbers. Get these wrong and every later instruction reads the wrong wire — so derive them from your routing table, not from another bus’s file.

```
// MSX BusSignals (16-bit address, separate Z80 strobes)
typedef union {
    struct {
        uint32_t addr:16;    // A0..A15  GP0..GP15
        uint32_t resv:4;
        uint32_t data:8;     // D0..D7   GP20..GP27
        uint32_t rd:1, wr:1, iorq:1, memrq:1; // GP28..GP31
    } __attribute__((packed));
    uint32_t combined;
} __attribute__((packed)) BusSignals;
```

9.2 Decode: the `wait_sel` decision

`wait_sel` encodes what “the host is talking to us” means, and the two examples sit at opposite ends of the difficulty range.

MSX — a ready-made select line. The cartridge slot hands the card a single `/SLTSL` that is asserted exactly when this slot is addressed. So `wait_sel` is almost nothing: wait for it, raise the IRQ, wait for it to release.

```
; MSX wait_sel (SLTSL pin configured inverted, so "wait 1" = /SLTSL low)
.program wait_sel
.wrap_target
    wait 1 gpio SLTSL_PIN [WAIT_CYCLES]    ; selected
    irq 0
    wait 0 gpio SLTSL_PIN                  ; deselected
.wrap
```

CoCo — decode it yourself. The cartridge port has two selects (`/CTS` for ROM, `/SCS` for the I/O spot) and no single “card selected” line, so the program reads the pins, masks the two select bits, and only proceeds when one is asserted — then qualifies on `R/W` and the `E` clock for timing.

```
; CoCo wait_sel (decode /CTS or /SCS, then time against E)
idle:
    mov osr, ~pins                ; all pins, inverted
    out x, NUM_PINS               ; grab the /CTS /SCS bits
    jmp !x idle                   ; both high -> not us
    jmp pin send                  ; R/W high (read)? no need to wait
    wait 1 gpio CLOCK_PIN         ; else wait for E
send:
    irq IRQ_SEL                   ; selected
    wait 0 gpio CLOCK_PIN         ; end of this bus cycle
```

The lesson is the decision, not the code: **does your bus give you a select line, or must you synthesise one from address-decode and strobes?** If it hands you one (MSX), `wait_sel` is trivial. If it does not (CoCo, and most backplane buses), you decode — either in PIO instructions, or by adding a little external logic that produces a single select line and reduces the problem to the MSX case.

9.3 Driving data: the `read` decision

`read` puts a byte on `D0-D7` when core 1 supplies one and tri-states the pins the rest of the time. The decision here is **what edge tells you the cycle is ending** so you know when to release the bus.

MSX keys off the select line itself: it loops checking the FIFO, drives the data while `/SLTSL` stays asserted (`jmp pin`), and returns the pins to inputs when the FIFO drains or the select releases.

CoCo keys off the `E` clock, using a side-set bit to flip `pindirs` in lockstep with the bus cycle:

```
; CoCo read (side-set flips data-pin direction around E)
.program read
.side_set 1 opt
    mov x, ~null side 1          ; D0-7 start as inputs
.wrap_target
    pull block                   ; byte from core 1
    out pins, DATA_WIDTH
    mov osr, ~null
    out pindirs, DATA_WIDTH side 0 ; drive D0-7
    wait 1 gpio CLOCK_PIN         ; through the E cycle
    wait 0 gpio CLOCK_PIN
    mov osr, null
    out pindirs, DATA_WIDTH side 1 ; release D0-7
.wrap
```

So the `read` decision follows directly from the bus’s timing model: a bus with a clean clock (CoCo’s `E`, or any synchronous backplane) hangs the direction flip on that clock; a bus framed by a chip-select (MSX) hangs it on the select. Identify your timing reference first and the `read` program writes itself.

`send_bus` carries no per-bus decision — it samples the GPIO and autopushes the 32-bit word — so it is copied unchanged between board files. Keep it verbatim.

9.4 Building the board into fujiversal

`fujiversal` selects a board file through CMake. Add yours to the RP2350 board set alongside the existing two:

```
# CMakeLists.txt — RP2350 board set
if(BOARD STREQUAL "msx_proto_260402"
  OR BOARD STREQUAL "coco_proto_260402"
  OR BOARD STREQUAL "<your_board>") # <-- new
  set(PICO_BOARD "pimoroni_pga2350" CACHE STRING "" FORCE)
  set(PICO_PLATFORM "rp2350-arm-s" CACHE STRING "" FORCE)
  set(PICO_CHIP "rp2350" CACHE STRING "" FORCE)
endif()
```

IMPORTANT

If your routing uses any GPIO above 31 (as both examples do — selects and reset live up there), the build **must** define `PICO_PIO_USE_GPIO_BASE=1` so the PIO can reach the upper bank. `setup_state_machine()` in `setup_sm.cpp` computes the GPIO base/range and rejects an illegal span (one straddling the 16↔32 boundary) — heed its return codes when a state machine silently fails to start.

9.5 Generating the host ROM

The bytes the host fetches from the emulated ROM come from `fujinet-config`, not `fujiversal`. Build the host image first, then bake it in:

```
# 1. build the host loader + CONFIG for your platform (fujinet-config)
make PLATFORM=<platform> # -> config-<platform>.rom (Chapter 15)

# 2. build the RP2350 firmware with that ROM
cd ../fujiversal
make ROM_FILE=../fujinet-config/config-<platform>.rom BOARD=<your_board>
# -> build/<your_board>/fujiversal_<your_board>.uf2
```

NOTE

Applying all of this to a bus that has **no** board file yet — decoding it from scratch, choosing the byte-pipe address, writing the union — is the design exercise in Chapter 19, worked through for the 8-bit ISA bus. It is the place to go once the MSX and CoCo files make sense.

CHAPTER 10

Bringing the RP2350 up

With a `.uf2` in hand, prove the bus interface in isolation before you ask the ESP32 to do anything.

10.1 Flash and enumerate

Hold the BOOTSEL button while plugging the Core2350B into USB; it mounts as a mass-storage drive. Copy the `.uf2` (or use `picotool load`). On reset the board should enumerate as a USB CDC-ACM serial device — that port is the link the ESP32 will later own. Open it from your workstation first; you have a debug console before the ESP32 is in the loop.

10.2 The loopback test

The cleanest first proof needs no host bus at all. With the RP2350 on USB and a terminal open on its CDC port, anything the firmware’s core 0 places in the RX ring appears on `GETC`, and anything written to `PUTC` is sent out the CDC port. So:

1. From the workstation terminal, send a byte. Confirm (with a logic analyzer on `J3 / J4`, or a tiny host test program) that a host read of `GETC` returns it and that `STATUS` showed `available` first.
2. Have the host write a byte to `PUTC`; confirm it arrives on the workstation terminal.

That round trip exercises both PIO directions, the multicore FIFO, and the USB bridge — layers 2 and 3 of `fig-boards` — without any FujiNet firmware running.

10.3 Debugging with the registers

The four registers are also your instrument. A short host program that spins reading `STATUS` and dumping `GETC` is the equivalent of a serial monitor for the byte pipe. When something later goes wrong end-to-end, the question “is the byte pipe healthy?” is answered here, below FujiBus and below the ESP32.

TIP

Build `fujiversal` with `USE_STDIO / VERBOSE_DEBUG` during bring-up to get `printf` over the CDC port, then turn it off: the debug build steals the same CDC channel the ESP32 needs, so a verbose RP2350 and a connected ESP32 cannot coexist. Bring the byte pipe up verbose, then go quiet and hand the port to the ESP32.

PART IV

The ESP32 Device Firmware

Where the work gets easy. On the ESP32 a tandem platform is a FujiBus serial transport, so it reuses the existing `rs232` bus, device, and media classes. This part shows the build target, the pin map, the device and media seams, the host ROM, and the client library.

CHAPTER 11

Where a bus platform fits in fujinet-firmware

The ESP32 never sees your bus — not MSX, not CoCo, not anything. It sees a CDC-ACM serial port carrying FujiBus packets. That is **exactly** what the firmware’s `rs232` bus already consumes — the same class used by the real RS-232 FujiNet. The two `fujiversal` build targets that already exist, `fujiversal-rs232` and `fujiversal-drivewire`, are proof of the pattern: they pair the RP2350 with the ESP32 over USB and select an existing bus on the ESP32 side.

11.1 The rs232 bus is the FujiBus transport

`lib/bus/rs232/` is misleadingly named: it is the FujiBus/FEP-004 engine, not an RS-232 UART driver. It contains its own copy of `FujiBusPacket.cpp` (the same encoder you met in Chapter 3) and a `systemBus` that reads and writes whole packets:

```
// lib/bus/rs232/rs232.h (abridged)
class systemBus {
    std::forward_list<virtualDevice *> _daisyChain;
    IOChannel *_port;
#if FUJINET_OVER_USB
    ACMChannel _serial;          // USB-CDC host channel  <-- the fujiversal path
#else
    UARTChannel _serial;        // a real UART
#endif
    std::unique_ptr<FujiBusPacket> readBusPacket(int first = -1);
    void writeBusPacket(FujiBusPacket &packet);
    void sendReplyPacket(fujiDeviceID_t source, bool ack,
                        const void *data, size_t length);
    void addDevice(virtualDevice *pDevice, fujiDeviceID_t device_id);
    // ...
};
extern systemBus SYSTEM_BUS;
```

The `FUJINET_OVER_USB` switch is the whole story: when the firmware is built to talk to the RP2350 over USB, `_serial` is an `ACMChannel` (a USB-CDC **host** channel); when it drives a physical serial port, it is a `UARTChannel`. A tandem bus platform uses the USB path, identical to `fujiversal-rs232`.

11.2 What this means for your effort

IMPORTANT

For a tandem bus-based platform, you usually write **no new bus class and no new device classes on the ESP32**. You add a build target and a pin map, and reuse `rs232`. New code on the ESP32 is needed only when your platform exposes a device the existing set does not, or needs a disk image format not already handled. Budget your engineering accordingly: the deep work is the PIO (Part III) and the host side (Chapters 15–16), not here.

CHAPTER 12

Adding the platform to the build system

PlatformIO drives the firmware build. Every platform is one `.ini` in `build-platforms/` plus a pin map. Model your target on `platformio-fujiversal-rs232.ini` — substitute your platform’s name for `<platform>` below.

12.1 The build platform file

```
; build-platforms/platformio-fujiversal-<platform>.ini
[fujinet]
build_bus      = RS232          ; reuse the FujiBus serial bus
build_platform = BUILD_RS232    ; ... and its device/media set

[env:fujiversal-<platform>]
build_type = debug
build_flags =
    ${env.build_flags}
    -D PINMAP_FUJIVERSAL_<PLATFORM>      ; <-- new pin map (below)
    -D CONFIG_USB_HOST_ENABLED=1         ; ESP32-S3 is the USB *host*
    -D CONFIG_USB_CDC_ACM_HOST_ENABLED=1
platform      = espressif32@${fujinet.esp32s3_platform_version}
platform_packages = ${fujinet.esp32s3_platform_packages}
board         = esp32-s3-wroom-1-n16r8
```

Three lines carry the design: `build_bus = RS232` chooses the FujiBus transport; `CONFIG_USB_HOST_ENABLED / CONFIG_USB_CDC_ACM_HOST_ENABLED` make the ESP32-S3 a USB host so it can open the RP2350’s CDC port; and `PINMAP_FUJIVERSAL_<PLATFORM>` selects your board’s pin assignments.

12.2 The pin map

Add a pin-map header guarded by `PINMAP_FUJIVERSAL_<PLATFORM>` and include it in the firmware’s pin-map dispatch. On a `fujiversal` board the pin map is small — the heavy bus I/O lives on the RP2350, so the ESP32 pin map mostly declares the USB-host data pins, the status LEDs, the button, and the SD pins of the Freenove module.

```
// include/pinmap/fujiversal_<platform>.h (mirror fujiversal_rs232.h)
#ifdef PINMAP_FUJIVERSAL_<PLATFORM>
#define PIN_LED_WIFI      GPIO_NUM_..    // white WiFi LED
#define PIN_LED_BUS       GPIO_NUM_..    // bus activity LED
#define PIN_BUTTON_A      GPIO_NUM_..
// USB host D+/D- and SD pins per the Freenove ESP32-S3-CAM board
// (no parallel-bus pins here: the RP2350 owns the host bus)
#endif
```

NOTE

The status-LED and button conventions are shared across FujiNet hardware (white = WiFi, an amber/orange = bus, Button A + a safe-reset). Match them so the existing firmware LED/boot logic “just works” — the `fujinet-firmware lib/hardware` LED code keys off these names.

12.3 Partitions and flashing

Reuse the existing 16 MB partition layout (`fujinet_partitions_16MB.csv`); the firmware image, the SPIFFS/FAT data partition (web UI, CONFIG assets), and OTA slots are platform-independent. Build and flash with the new env:

```
pio run -e fujiversal-<platform>
pio run -e fujiversal-<platform> -t upload    # over the ESP32-S3 USB/JTAG port
```

CHAPTER 13

Device classes

A FujiNet “device” is a class derived from `virtualDevice` that handles FujiBus packets for one device ID. The `rs232` device set you inherit covers the whole standard FujiNet feature list.

13.1 What you inherit

lib/device/rs232/		
0x70	rs232Fuji	Mounts, host slots, app-keys, hashing, adapter config — the CONFIG back end
0x31 — 0x3F	rs232Disk + media	Virtual disk drives
0x71 — 0x78	rs232Network	The N: device — one per unit, 8 units
0x40 — 0x43	rs232Printer	Printer emulation to PDF
0x50 — 0x53	rs232Modem	Modem / serial passthrough
0x45	clock	APETime real-time clock
0x5A	rs232CPM	CP/M console

Table 14. The device classes a `BUILD_RS232` platform gets for free. Each is a `virtualDevice` registered with `SYSTEM_BUS.addDevice()`.

13.2 The virtualDevice contract

If you ever do need a new device, the interface is small. Every device implements two pure-virtual methods that the bus calls:

```
// lib/bus/rs232/rs232.h
class virtualDevice {
protected:
    fujiDeviceID_t _devnum;
    // Per-command dispatch: usually a switch() on packet.command()
    virtual void rs232_process(FujiBusPacket &packet) = 0;
    // Return 4 status bytes (the historical DVSTAT semantics)
    virtual void rs232_status(FujiStatusReq reqType) = 0;
    virtual void shutdown() {}
public:
    fujiDeviceID_t id() { return _devnum; }
    bool is_config_device = false; // true for the disk that boots CONFIG
    bool device_active = true;
};
```

The bus reads a packet, finds the device whose `id()` matches `packet.device()`, and calls its `rs232_process()`. To add a device you subclass `virtualDevice`, implement those two methods, and register an instance:

```
SYSTEM_BUS.addDevice(new myDevice(), FUJI_DEVICEID_SOMETHING);
```

NOTE

Devices are registered during `SYSTEM_BUS.setup()`, which the platform’s `BUILD_*` selection wires up. For a stock build you change nothing here — the `rs232` set registers itself.

CHAPTER 14

Media classes

A media class turns a host disk-image file on the SD card into the sectors a virtual disk device serves. They live in `lib/media/`, one directory per platform (`atari`, `apple`, `adam`, `cbm`, `mac`, `drivewire`, `rs232`, ...), behind the `MediaType` interface in `lib/media/media.h`.

14.1 Reuse first

The disk device (`rs232Disk`) presents **block** storage: the host reads and writes 512-byte sectors by number (the FujiBus disk command carries the sector as a 32-bit `C1234` field). If your platform's software is happy with raw block images, the existing `rs232` media path already serves them and you add nothing.

14.2 When you need a new media class

Add a `lib/media/<platform>/` only when the host expects an **image format** with structure the firmware must understand — a header, an interleave, a copy-protection scheme, a non-512 sector size. The class implements:

```
// the shape of a MediaType (lib/media/media.h)
class MediaType {
public:
    virtual bool read(uint32_t blockNum, uint16_t *readcount) = 0;
    virtual bool write(uint32_t blockNum, bool verify) = 0;
    virtual bool format(uint16_t *responsesize) = 0;
    virtual mediatype_t mount(fnFile *f, uint32_t disksize) = 0;
    virtual void unmount() = 0;
    static mediatype_t discover_disktype(const char *filename); // by extension
};
```

Decide what your host's software expects: a flat sector image (a `.dsk` or `.img` of a floppy or hard disk) maps directly onto block reads and needs no new class. A structured format — an interleave, an on-disk header, a copy-protected original — would. Start flat; add a `MediaType` only when a real image format forces you to.

TIP

`discover_disktype()` dispatches on file extension. When you do add a format, register its extension there so a user mounting `disk.img` from CONFIG gets the right `MediaType` automatically.

CHAPTER 15

The host ROM and CONFIG

The emulated ROM the RP2350 serves is built by `fujinet-config`. This is the code that runs **on the host CPU**, and it is as platform-specific as the PIO — it is written in the host’s assembly/C and it drives the byte pipe directly.

15.1 What fujinet-config produces

`fujinet-config` builds, per platform, two things baked into one ROM image:

1. A **loader**: the few hundred bytes the host runs at boot, in whatever form the machine autostarts (the boot-device goal from Chapter 1). MSX wants a cartridge ROM carrying the `AB` header at `0x4000`; CoCo wants an autostart Program Pak at `0xC000`; a PC ISA card wants an option ROM (`0x55 0xAA` , length, entry point) the BIOS calls during its expansion-ROM scan. Whatever the form, the loader brings up the byte pipe, asks FujiNet to mount the configured boot disk, and either boots it or launches CONFIG.
2. The **CONFIG application**: the full-screen UI for choosing WiFi hosts, browsing TNFS, and mounting images into the eight disk slots. Its screen text for each platform lives in `fujinet-config/src/<platform>/screen.c` and is rendered in that machine’s native character set.

15.2 How the loader talks to FujiNet

The loader uses the same four registers as everything else. In pseudocode, sending one FujiBus byte and reading the reply is:

```
put_byte(b):    while (inb(IO_BASE+IO_STATUS) & BUSY) ;    ; optional flow ctl
                outb(IO_BASE+IO_PUTC, b)
get_byte():     while (!(inb(IO_BASE+IO_STATUS) & IO_FLAG_AVAIL)) ;
                return inb(IO_BASE+IO_GETC)
```

On ISA those `inb` / `outb` are 8088 `IN` / `OUT` instructions to ports `0x300 – 0x303`. On MSX they are memory reads/writes at `0xBFFC`; on CoCo, at `0xFF41`. **Only the access method and the addresses change** — the protocol above is identical, which is exactly why the client library (next chapter) is mostly shared C with a thin per-platform shim.

15.3 Building and chaining the ROM

```
# in fujinet-config
make PLATFORM=<platform>    # builds loader + CONFIG -> config-<platform>.rom
```

The output is consumed by `fujiversal` (Chapter 9, `ROM_FILE=...`), which converts it to a C array (`build/<board>/rom.h`) and serves it from `BUS_ROM_BASE`. Change the host UI, rebuild `fujinet-config`, rebuild `fujiversal`, reflash the RP2350.

CHAPTER 16

The client library

`fujinet-lib` is the C library application programmers link against to use FujiNet — the same one CONFIG and every networked program use. The experimental tree, `fujinet-lib-experimental`, is the FujiBus-native version, and its structure is the template for your platform's port.

16.1 The backend pattern

```
fujinet-lib-experimental/  
  include/      fujinet-bus.h, fujinet-bus-ezcall.h, FUJI_FIELD_*, FujiDCB  
  common/      network.c, network_json.c, fuji_*.c   (platform-independent)  
  bus/  
    apple2/    atari/   c64/   coco/   msx/   msdos/   adam/      (one dir per platform)  
    <platform>/  <-- you add this  
  Makefile      PLATFORMS = coco apple2 atari c64 msx msdos adam  (+ yours)
```

`common/` is the bulk of the library and is shared verbatim. Each `bus/<platform>/` provides the same small surface: the three transport primitives and the platform's port I/O.

16.2 What bus/

/ contains

`portio.s` / `portio.h`

The platform's access to the four byte-pipe registers — `IN` / `OUT` to an I/O port (the `0xBFFC`-style memory window on MSX, an I/O port on a PC), or loads/stores to a memory address.

`fujinet-bus-<platform>.c`

Implements `fuji_bus_call`, `fuji_bus_read`, `fuji_bus_write`: build the FujiBus packet (header, descriptors, AUX, payload, checksum), SLIP-encode it, stream it out `PUTC`, then read the SLIP reply back through `GETC`.

Table 15. The two pieces of a `fujinet-lib` bus backend. The existing `bus/msdos/portio.s` (real PC `IN` / `OUT`) and `bus/msx/portio.s` (memory-mapped at `0xBFFC`) are the two shapes to crib from.

The public surface the rest of the library calls is just:

```
// include/fujinet-bus.h  
bool fuji_bus_call(uint8_t device, uint8_t fuji_cmd, uint8_t fields,  
                  uint8_t aux1, uint8_t aux2, uint8_t aux3, uint8_t aux4,  
                  const void *data, size_t data_length,  
                  void *reply, size_t reply_length);  
size_t fuji_bus_read (uint8_t device, void *buffer, size_t length);  
size_t fuji_bus_write(uint8_t device, const void *buffer, size_t length);
```

and the `FUJICALL_*` convenience macros wrap it with the right `FUJI_FIELD_*` descriptor. A program that opens an N: URL never sees a byte of FujiBus framing; it calls `network_open()` in `common/`, which calls `fuji_bus_call()`, which calls your `bus/<platform>` primitives, which hit the byte pipe.

16.3 Building it

Add your platform to `PLATFORMS` in the Makefile and build:

```
make <platform>    # builds fujinet.lib for your backend
```

`SRC_DIRS = common bus/%PLATFORM%` already composes the shared core with your new backend; no other Makefile change is needed.

TIP

Bring the library up against the **loopback** of Chapter 10 before the full firmware is ready: a host program that calls `fuji_bus_write()` and watches the bytes appear on the RP2350's USB console proves your SLIP encoder and port I/O independent of the ESP32. Then connect the ESP32 and the same call returns a real `ACK`.

PART V

Integration & Validation

Putting the layers together: the milestone ladder from first power-on to a booting machine, a troubleshooting matrix indexed by symptom, and a worked design exercise that applies the whole method to a bus nobody has built yet.

CHAPTER 17

The bring-up milestone ladder

Bring a platform up in the order the layers stack, lowest first. Each rung is independently testable, and a failure is contained to the rung you are on. Do not move up until the current rung is solid.

0	Boards seated, power correct	Continuity passes; nothing warms up on USB power; <code>J9</code> set for bench.
1	RP2350 enumerates	The Core2350B appears as a USB CDC device on your workstation.
2	ESP32 alive	Firmware flashed (<code>pio run -e fujiversal-<platform> -t upload</code>); WiFi joins; SD mounts.
3	Byte pipe loopback	A host <code>GETC</code> returns a byte you injected; a host <code>PUTC</code> reaches the USB console (Chapter 10).
4	Address decode	The logic analyzer shows the PIO IRQ firing only when the host actually addresses the interface — and on no other bus cycle.
5	FujiBus ACK	A <code>fuji_bus_call()</code> from the host library returns <code>ACK</code> from the ESP32 (the Fuji device answers an adapter-config query).
6	CONFIG boots	The host runs the boot ROM (cartridge / Program Pak / option ROM), the loader runs, and CONFIG draws on screen.
7	Mount + boot a disk	CONFIG mounts a <code>.img</code> ; the machine boots it through the disk device.
8	<code>N:</code> works	A program opens <code>N:HTTP://...</code> and reads data over WiFi.

Table 16. The milestone ladder for a tandem bring-up. Milestones 0–4 are hardware/PIO (Parts II–III); 5–8 are firmware/host (Part IV).

NOTE

Milestones 1–3 are exactly the `fujinet-bringup` MVP (Chapter 1): a minimal byte relay plus `iotest` proving two-way communication. They need **no host bus board at all** — they run on the bench over USB with the micro wired to your machine’s bus, and you can reach milestone 3 before the bus adapter PCB even arrives. Start there, with the FujiNet firmware as a PC build, and order your work so the slow-to-fabricate adapter is never on the critical path. Milestone 5 is the `fujinet-bringup` “Hello World” (fetch the adapter config) succeeding for the first time.

CHAPTER 18

Troubleshooting

Indexed by symptom. The “layer” column points you at the part of this guide — and the part of `fig-boards` — that owns the fault.

Machine hangs or crashes the moment the interface is connected	1	The decode is too loose — it responds to cycles that are not ours. (On ISA, forgetting to gate I/O decode on <code>AEN</code> low is the classic case; the card then answers DMA addresses.) Tighten the <code>wait_sel</code> condition (Ch. 9). Or a signal is mis-routed and two drivers are fighting — check your jumper/patch routing.
Interface does nothing; no PIO IRQ on the analyzer	2	<code>wait_sel</code> decode never matches. Wrong <code>IO_BASE</code> , wrong strobe polarity (strokes are usually active-low), or a needed signal not routed to its GPIO. Probe <code>J2</code> vs <code>J3</code> / <code>J4</code> .
PIO IRQs fire on every bus cycle	2	<code>wait_sel</code> is not pre-qualifying the address window — it selects on the strobe alone. Add the window decode, or feed a decoded select line from adapter logic (Ch. 9).
Host reads our register but gets <code>0xFF</code> / garbage	2–3	Data not driven in time, or direction backwards. Check the <code>read</code> program's <code>pindir</code> flip timing against your timing reference — the clock (CoCo) or the select line (MSX) (Ch. 9).
Bytes go out but no reply ever comes back	3–4	Loopback (M3) first. If loopback is fine, the ESP32 isn't opening the CDC port (<code>CONFIG_USB_CDC_ACM_HOST_ENABLED</code> ?) or the RP2350 is still in verbose <code>printf</code> mode stealing the channel (Ch. 10).
<code>fuji_bus_call</code> returns failure though bytes flow	4	Framing bug: checksum (the add-with-fold, not XOR), little-endian <code>length</code> , or a descriptor/AUX mismatch. Diff your encoder against <code>FujiBusPacket.cpp</code> (Ch. 3, 16).
Host never runs the boot ROM	5	The ROM is not where/what the machine autostarts from — wrong <code>BUS_ROM_BASE</code> , or a bad header (the MSX <code>AB</code> signature, the CoCo autostart bytes, the PC option-ROM <code>0x55 0xAA</code> /length/checksum). Verify the served bytes at the ROM window with the analyzer on the memory-read

Table 17: Troubleshooting matrix. The layer numbers match `fig-boards`; flicker is detected if any of the missing decoupling at the adaptor.

CHAPTER 19

Design exercise: a bus nobody has built yet

The two worked examples were real — code you can open in the repository. This closing chapter is deliberately different: it applies the same method to a bus that has **no** board file, no adapter, and no firmware yet — the 8-bit IBM PC/XT ISA bus — purely as a reasoning exercise.

IMPORTANT

Everything here is **illustrative and unbuilt**. The addresses, the PIO sketch, and the build names are a worked design, not a proven port. Its value is the reasoning, not the numbers. Read it as “how you would think this through,” and verify every choice on a logic analyzer if you ever build it.

19.1 The method: what changes from one bus to the next

Everything platform-specific is captured by a short diff. Walk it for any new bus — ISA below, or the 6502 cartridge port, S-100 backplane, or Apple slot on your bench — and you have your design. Everything **not** in this table is shared and unchanged.

Disk interface, or not	Decision 1 (Ch. 1): a disk interface to ride, or FEP-004 direct. Favour whatever boots from bare metal.
Connector and voltage	The adapter (Ch. 6–7); on an RP2350, voltage is usually nothing to do.
Signal-to-GPIO routing	The jumpers and patches (Ch. 4–6): which signals land on the default map, which you reroute, which you wire in from the breakouts.
The decode rule	<code>wait_sel</code> (Ch. 9): what “selected” means, and the timing reference (clock edge, chip-select, or strobe).
Data-direction timing	The <code>read</code> program (Ch. 9): which edge flips <code>pindir</code> .
Byte-pipe + ROM address	<code>IO_BASE</code> / <code>BUS_ROM_BASE</code> and the register offsets — wherever the host can reach four registers and present a boot ROM.
Host loader + screens	<code>fujinet-config/src/<platform>/</code> — the access method and the CONFIG character set.
Client port I/O	<code>fujinet-lib bus/<platform>/portio.*</code> — memory- or port-mapped.

Table 18. The per-bus diff. FujiBus framing, the ESP32 device and media classes, the `N`: stack, and the byte-pipe model never appear here — they do not change.

19.2 Working the diff for ISA

Disk interface? No. A PC/XT has no peripheral disk-serial bus to ride, so FujiNet speaks FEP-004 directly. **Boot device?** Yes, and the PC offers a clean way: the BIOS scans the expansion-ROM region `0xC0000` — `0xDFFFF` at power-on for option ROMs marked `0x55 0xAA`, a length byte, and an entry point. Present one at, say, `0xC8000` and FujiNet boots from bare metal — the goal met.

The decode rule — and the trap. ISA gives you no ready-made select line, so this is the CoCo-style “decode it yourself” case with one extra hazard: `AEN`. The condition for an I/O cycle being **ours** is “address in our port window AND a command strobe (`/IOR` / `/IOW`) asserted AND `AEN` low”. `AEN` is high during DMA; an I/O interface that ignores it answers DMA addresses and crashes the machine. The ROM window is simpler — address in `0xC8000...` AND `/MEMR` — and ignores `AEN`.

Signal routing — the one place ISA is easy. ISA is the bus the board’s default routing was drawn from, so it lands almost untouched: `A0 – A19` on `GP0 – GP19`, `D0 – D7` on `GP20 – GP27`, the four strobes on `GP28 – GP31`, `AEN` on `GP32`. One caveat to reason about: `send_bus` samples only `GP0 – GP31`, so `AEN` at `GP32` sits just past the latched word — gate it inside `wait_sel` (which can `wait` on any GPIO) rather than reading its level in core 1.

Data direction. There is no single convenient clock to key on, so flip `pindirs` on the active strobe edge (`/IOR` for the port read, `/MEMR` for the ROM) — the strobe is the timing reference.

Byte pipe + ROM. Four I/O ports at `0x300 – 0x303` (the conventional home-brew card range) for `GETC / STATUS / PUTC / CONTROL`, and the option ROM at `0xC8000`. The pinout that supports all of this is Appendix B.

19.3 What you would build next

If the design survives the bench — milestones 0–5 of Chapter 17, reached with `fujinet-bringup` and a logic analyzer — the artifacts are the familiar five, and **only** these five:

<code>boards/isa_proto.pio</code>	The decode + byte pipe (the <code>wait_sel / read</code> reasoning above).
<code>fujiversal</code> board entry	A CMake board + <code>PICO_PIO_USE_GPIO_BASE=1</code> .
<code>platformio-fujiversal-isa.ini</code>	An ESP32 build target — <code>build_bus = RS232</code> , USB host.
<code>fujinet-config/src/isa/</code>	The option-ROM loader + CONFIG screens.
<code>fujinet-lib/bus/isa/</code>	<code>IN / OUT portio</code> + the FujiBus encoder.

Table 19. Everything a new ISA platform would add. Nothing in the FujiBus, device, media, or `n:` layers changes — which is the entire payoff of the tandem design, and the note to end on.

TIP

The fastest possible bring-up of any brand-new bus: feed the PIO a single decoded “selected” line from a small GAL on the adapter (so `wait_sel` reduces to the MSX form), put the byte pipe wherever the host can do four `peek / poke` s, and reuse `BUILD_RS232` whole. You can have a machine exchanging FujiBus packets before you have written one line of decode logic in PIO — which is exactly the `fujinet-bringup` philosophy, all the way down.

APPENDIX A

FujiBus quick reference

Frame: `END` (`0xC0`) · header · descriptors · AUX (little-endian) · payload · `END` . SLIP escapes: `0xC0` → `0xDB 0xDC` , `0xDB` → `0xDB 0xDD` .

Header (6 bytes): `device` , `command` , `length` (u16 LE, total incl. header), `checksum` (add-with-carry-fold, this byte zeroed during calc), `descr` (first field descriptor).

Field descriptors (`descr & 0x07` ; bit 7 = more follow):

0 none	1 1×u8	2 2×u8	3 3×u8	4 4×u8	5 1×u16	6 2×u16	7 1×u32
-----------	-----------	-----------	-----------	-----------	------------	------------	------------

Replies: `command` = `ACK` `0x06` (success, optional payload) or `NAK` `0x15` (failure).

Device IDs: `0x31` – `0x3F` disk · `0x40` – `0x43` printer · `0x45` clock · `0x50` – `0x53` serial · `0x5A` CP/M · `0x70` Fuji control · `0x71` – `0x78` network · `0x99` MIDI · `0xFF` bus controller (DBC, intercepted by the RP2350).

Common commands (`fujiCommandID.h` is authoritative; high range is the Fuji control device):

<code>0x4F</code>	<code>'O'</code>	<code>OPEN</code>	<code>0xF8</code>	<code>MOUNT_IMAGE</code>
<code>0x52</code>	<code>'R'</code>	<code>READ</code>	<code>0xF9</code>	<code>MOUNT_HOST</code>
<code>0x57</code>	<code>'W'</code>	<code>WRITE</code>	<code>0xF7</code>	<code>OPEN_DIRECTORY</code>
<code>0x53</code>	<code>'S'</code>	<code>STATUS</code>	<code>0xF6</code>	<code>READ_DIR_ENTRY</code>
<code>0x43</code>	<code>'C'</code>	<code>CLOSE</code>	<code>0xE8</code>	<code>GET_ADAPTERCONFIG</code>
<code>0x50</code>	<code>'P'</code>	<code>PARSE / PUT</code>	<code>0xD9</code>	<code>CONFIG_BOOT</code>
<code>0x51</code>	<code>'Q'</code>	<code>QUERY</code>	<code>0x80</code>	<code>JSON_PARSE</code>
<code>0x06</code>		<code>ACK</code>	<code>0x81</code>	<code>JSON_QUERY</code>
<code>0x15</code>		<code>NAK</code>	<code>0xFF</code>	<code>RESET</code>

Table 20. A working subset of FujiBus commands. The full enum (≈150 names, with per-device aliases) is `fujiCommandID.h` , shared by firmware and lib.

APPENDIX B

ISA 8-bit (PC/XT) 62-pin pinout

Reference for the Chapter 19 design exercise: the standard 8-bit ISA edge, with the board's default GPIO routing for the signals FujiNet uses (from `tbl-gpmap`). Pin rows: A = component side, B = solder side.

A1	/I/O CH CK	—
A2–A9	D7 ... D0	27 ... 20
A10	I/O CH RDY	—
A11	AEN	32
A12–A31	A19 ... A0	19 ... 0

Table 21. A side (data, address, `AEN`, ready).

B1	GND	—
B2	RESET DRV	35
B3	+5V	—
B11	/SMEMW	29
B12	/SMEMR	28
B13	/IOW	31
B14	/IOR	30
B20	CLK	33
B28	ALE	34
B30	OSC	—
B4..B26	IRQ2–7, DRQ/DACK	—
B5/B7/B9	–5V / –12V / +12V	—

Table 22. B side (power, strobes, clocks, IRQ/DMA).

NOTE

The schematic's net names confirm this mapping: `~{SMEMR}` / `~{SMEMW}`, `~{IOR}` / `~{IOW}`, `AEN`, `ALE`, `CLK`, `OSC`, `IO_READY`, `IRQ2 – IRQ7`, `DRQ1 – DRQ3`, `~{DACK0}` – `~{DACK3}`, `TC`, and `BA00 – BA19` (buffered address). FujiNet uses only the boxed subset above.

APPENDIX C

Universal board jumper & test-point reference

JP1 — JP36	solder, bridged	One per bus signal; cut to isolate or to insert a buffer.
JP37 — JP39	solder, open	Optional configuration straps.
J1	Bus_ISA_8bit	Universal bus header — adapter mates here.
J2	2×13 header	ISA-side breakout (logic-analyzer tap).
J3 , J4	2×13 / headers	RP2350 GPIO breakouts.
J5 — J8	1×20	Core2350B and ESP32-S3 module seats.
J9	2×2	Power-source selection.
U1	module	WaveShare Core2350B (RP2350B).
U2	module	Freenove ESP32-S3-CAM.
D1	diode	Power-rail protection.

Table 23. Reference designators on `Universal-proto-v1` . There are no buffer ICs; add level-shifting on the adapter (Ch. 6).

APPENDIX D

Repository map

Where every artifact in this guide lives.

<code>fujinet-bringup/README.md</code>	Start here. The bring-up-first method (relay + <code>iotest</code> + PC firmware).
<code>fujinet-bringup/iotest/</code>	Host two-way-comms test + per-platform <code>portio</code> examples (14 platforms).
<code>fujinet-bringup/esp32/ , .../rp2350/</code>	Minimal byte-relay firmware (GPIO bus to USB serial).
<code>fujiversal/main.cpp</code>	Core 0 USB bridge + core 1 <code>romulan()</code> bus loop + DBC handler.
<code>fujiversal/boards/*.pio</code>	Per-board PIO + pin defines + <code>BusSignals</code> union (<code>msx_proto</code> , <code>coco_proto</code> ; add yours).
<code>fujiversal/setup_sm.cpp</code>	Generic PIO state-machine setup helper.
<code>fujiversal/FujiBusPacket.*</code>	FujiBus encoder/decoder (RP2350 copy).
<code>fujiversal/CMakeLists.txt</code>	Board selection, <code>PICO_PIO_USE_GPIO_BASE</code> .
<code>fujiversal-pcb-prototype/Bus-proto/</code>	<code>Universal-proto-v1</code> board (jumpers, breakouts).
<code>fujiversal-pcb-prototype/*-adapter/</code>	The <code>MSX-adapter</code> and <code>CoCo-adapter</code> (templates for yours).
<code>fujiversal-pcb-prototype/parts.pretty/ISA_8bit*</code>	ISA edge footprints.
<code>fujinet-firmware/build-platforms/platformio-fujiversal-*.ini</code>	Build targets (<code>rs232</code> , <code>drivewire</code> ; add yours).
<code>fujinet-firmware/lib/bus/rs232/</code>	FujiBus transport + <code>systemBus</code> (reused as-is).
<code>fujinet-firmware/lib/device/rs232/</code>	Device classes (Fuji, disk, network, printer, ...).
<code>fujinet-firmware/lib/media/</code>	Image formats (<code>MediaType</code>).
<code>fujinet-firmware/include/pinmap/</code>	Pin maps (add <code>fujiversal_<platform>.h</code>).
<code>fujinet-lib-experimental/bus/<plat>/</code>	Per-platform transport + port I/O (add <code>bus/<platform>/</code>).
<code>fujinet-lib-experimental/common/</code>	Shared <code>network</code> , <code>json</code> , <code>fuji</code> code.
<code>fujinet-config/src/<plat>/</code>	Host loader + CONFIG screens (add <code>src/<platform>/</code>).

Table 24. The file map. “Add ...” marks the artifacts a new platform creates; everything else is reused.

APPENDIX E

Glossary

AEN Address Enable. High during ISA DMA; an I/O card must decode only when it is low.

Byte pipe the four I/O registers (`GETC` , `STATUS` , `PUTC` , `CONTROL`) through which the host streams bytes to and from the RP2350.

DBC device ID `0xFF` , the bus controller (RP2350) itself; DBC-addressed packets are consumed locally for ROM bank-switching.

FEP-004 the FujiNet protocol proposal that FujiBus implements.

FujiBus the SLIP-framed packet protocol the host and ESP32 exchange.

fujiversal the RP2350 firmware that emulates the bus interface.

Option ROM a BIOS-scanned expansion ROM (`0x55 0xAA` header) in `0xC0000` — `0xDFFFF` ; FujiNet's boot loader on ISA.

PIO the RP2350's Programmable I/O — deterministic state machines that implement the bus timing.

RAMROM the RP2350's swappable emulated-ROM image, bank-switched via DBC commands.

Tandem design the ESP32 + RP2350 pairing for bus-based platforms.

APPENDIX F

Bill of materials (one bring-up rig)

1	Waveshare Core2350B (RP2350B)	U1 ; ≥ 40 usable GPIO
1	Freenove ESP32-S3-CAM (dual-USB, microSD)	U2 ; may need SD-pin ground mod
1	Universal-proto-v1 PCB + headers	fujiversal-pcb-prototype
1	Bus adapter PCB / patched header	you make (Ch. 6)
1–2	74LVC245 (data) + 74LVC buffers (addr/strobe)	for the buffered adapter
1	GAL/ATF16V8 (optional)	card-select decode for wait_sel
—	0.1 μF + 10 μF decoupling, headers, jumper wire	
1	USB hub with per-port power	reflash without disturbing the bus
1	Logic analyzer (≥ 8 ch, ≥ 24 MS/s)	J2 / J3 / J4 probing
1	Bus extender or socketed breakout	spares a fragile machine connector during bring-up

Table 25. What it takes to bring up one tandem board. The two dev boards and the proto board reach milestone 3 on their own.

FujiNet Platform Bring-Up Guide — Revision 3, June 2026.

Built with Typst from sources in fujinet-bringup , fujiversal , fujiversal-pcb-prototype , fujinet-firmware , fujinet-lib-experimental , and fujinet-config .

The network is as easy as the disk drive — once the bus says so.